

1. Realization of singly linked list

```
#include <stdio.h>

#include <stdlib.h>

struct node
{
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;

NODEPTR list = NULL;

NODEPTR p, q, temp;

NODEPTR getnode();
void insert_left(int x);
void insert_end(int x);
void insert_after(NODEPTR p, int x);
void delete_left();
void delete_last();
void delete_after(NODEPTR p);
void display();

int main()
{
    int ch, x, pos, i;
    char cont;
    do
    {
```

```
printf("\n--- Singly Linked List Menu ---");
printf("\n1. Insert at Left");
printf("\n2. Insert at Last");
printf("\n3. Insert After a Given Position");
printf("\n4. Delete from Left");
printf("\n5. Delete from Last");
printf("\n6. Delete After a Given Position");
printf("\n7. Display\n");
scanf("%d", &ch);
switch (ch)
{
case 1:
    printf("Enter element: ");
    scanf("%d", &x);
    insert_left(x);
    break;
case 2:
    printf("Enter element: ");
    scanf("%d", &x);
    insert_end(x);
    break;
case 3:
    printf("Enter position: ");
    scanf("%d", &pos);
    printf("Enter element: ");
    scanf("%d", &x);
```

```

    p = list;
    for (i = 1; i < pos && p != NULL; i++)
        p = p->next;
    insert_after(p, x);
    break;
case 4:
    delete_left();
    break;
case 5:
    delete_last();
    break;
case 6:
    printf("Enter position: ");
    scanf("%d", &pos);
    p = list;
    for (i = 1; i < pos && p != NULL; i++)
        p = p->next;
    delete_after(p);
    break;
case 7:
    display();
    break;
default:
    printf("Invalid choice\n");
}
printf("\nDo you want to continue (y/n): ");

```

```

        scanf(" %c", &cont); /* notice the space before %c */
    } while (cont != 'n' && cont != 'N');

    return 0;
}

NODEPTR getnode()
{
    return (NODEPTR)malloc(sizeof(struct node));
}

void insert_left(int x)
{
    p = getnode();
    p->info = x;
    p->next = list;
    list = p;
}

void insert_end(int x)
{
    p = getnode();
    p->info = x;
    p->next = NULL;

    if (list == NULL)
        list = p;
    else
    {

```

```

        for (q = list; q->next != NULL; q = q->next)
            ;
        q->next = p;
    }
}

```

```

void insert_after(NODEPTR p, int x)

```

```

{
    NODEPTR q;
    if (p == NULL)
        printf("Invalid insertion\n");
    else
    {
        q = getnode();
        q->info = x;
        q->next = p->next;
        p->next = q;
    }
}

```

```

/* Delete from left */

```

```

void delete_left()

```

```

{
    if (list == NULL)
        printf("Invalid deletion\n");
    else
    {

```

```
    p = list;
    list = p->next;
    printf("Deleted element is %d\n", p->info);
    free(p);
}
```

```
void delete_last()
```

```
{
    if (list == NULL)
    {
        printf("Invalid deletion\n");
        return;
    }
```

```
    p = list;
    q = NULL;
```

```
    while (p->next != NULL)
```

```
    {
        q = p;
        p = p->next;
    }
```

```
    if (q != NULL)
        q->next = NULL;
```

```
    else
        list = NULL;
```

```

    printf("Deleted element is %d\n", p->info);
    free(p);
}
void delete_after(NODEPTR p)
{
    NODEPTR q;
    int x;
    if (p == NULL || p->next == NULL)
        printf("Invalid deletion\n");
    else
    {
        q = p->next;
        x = q->info;
        p->next = q->next;
        q->next = NULL;
        free(q);
        printf("Deleted element is %d\n", x);
    }
}
void display()
{
    p = list;
    if (p == NULL)
        printf("List is empty\n");
    else
    {

```

```

while (p != NULL)
{
    printf("%d -> ", p->info);

    p = p->next;
}
printf("NULL\n");
}
}

```

2. Realization of doubly linked list

```

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int info;
    struct node *left;
    struct node *right;
};

typedef struct node* NODEPTR;

NODEPTR list = NULL;
NODEPTR p, q, temp;

NODEPTR getnode();
void insert_left(int x);
void insert_last(int x);
void insert_right(NODEPTR p, int x);

```



```

void insert_left_node(NODEPTR p, int x);
void delete_left();
void delete_last();
void delete_node(NODEPTR p);
void display();

int main()
{
    int c, x, pos, i;
    char ch;

    do
    {
        printf("\n--- Doubly Linked List Menu ---");
        printf("\n1. Insert at Left");
        printf("\n2. Insert at Right (End)");
        printf("\n3. Insert Right of a Given Position");
        printf("\n4. Insert Left of a Given Position");
        printf("\n5. Delete from Left");
        printf("\n6. Delete from Right");
        printf("\n7. Delete at a Given Position");
        printf("\n8. Display\n");

        scanf("%d", &c);

        switch (c)
        {
            case 1:
                printf("Enter element to insert at left: ");
                scanf("%d", &x);
                insert_left(x);
                break;
            case 2:
                printf("Enter element to insert at right: ");
                scanf("%d", &x);
                insert_last(x);
                break;

```

case 3:

```
printf("Enter position: ");
scanf("%d", &pos);
printf("Enter element: ");
scanf("%d", &x);
p = list;
for(i=1; i<pos && p!=NULL; i++)
    p = p->right;
insert_right(p, x);
break;
```

case 4:

```
printf("Enter position: ");
scanf("%d", &pos);
printf("Enter element: ");
scanf("%d", &x);
p = list;
for(i=1; i<pos && p!=NULL; i++)
    p = p->right;
insert_left_node(p, x);
break;
```

case 5:

```
delete_left();
break;
```

case 6:

```
delete_last();
break;
```

case 7:

```
printf("Enter position to delete: ");
scanf("%d", &pos);
p = list;
for(i=1; i<pos && p!=NULL; i++)
    p = p->right;
delete_node(p);
break;
```

case 8:

```
display();
```

```

        break;
    default:
        printf("Invalid choice\n");
    }

    printf("\nDo you want to continue (y/n): ");
    scanf(" %c",&ch);
} while(ch!='n' && ch!='N');
return 0;
}

```

```

NODEPTR getnode()
{
    return (NODEPTR)malloc(sizeof(struct node));
}

```

```

void insert_left(int x)
{
    temp=getnode();
    temp->info=x;
    temp->left=NULL;
    if(list==NULL)
    {
        temp->right=NULL;
        list=temp;
    }
    else
    {
        temp->right=list;
        list->left=temp;
        list=temp;
    }
    return;
}

```

```

void insert_last(int x)
{

```

```

p=getnode();
p->info=x;
p->right=NULL;
if(list==NULL)
{
    list=p;
    list->left=NULL;
}
else
{
    temp=list;
    while(temp->right!=NULL)
        temp=temp->right;
    temp->right=p;
    p->left=temp;
}
return;
}

void insert_right(NODEPTR p, int x)
{
    NODEPTR q, r;
    if(p==NULL)
    {
        printf("Invalid insertion\n");
        return;
    }
    q = getnode();
    q->info = x;
    r = p->right;
    q->left = p;
    q->right = r;
    p->right = q;
    if(r != NULL)
        r->left = q;
}

```

```

void insert_left_node(NODEPTR p, int x)
{
    NODEPTR q, l;
    if(p==NULL)
    {
        printf("Invalid insertion\n");
        return;
    }
    q = getnode();
    q->info = x;
    l = p->left;
    q->right = p;
    q->left = l;
    p->left = q;
    if(l != NULL)
        l->right = q;
    else
        list = q;
}

```

```

void delete_left()
{
    if(list==NULL)
    {
        printf("Invalid deletion\n");
        return;
    }
    p=list;
    if(p->right==NULL)
        list=NULL;
    else
    {
        list=p->right;
        list->left=NULL;
    }
    printf("%d\n",p->info);
    p->right=NULL;
}

```

```

    free(p);
}

void delete_last()
{
    if(list==NULL)
    {
        printf("Invalid deletion\n");
        return;
    }
    temp=list;
    while(temp->right!=NULL)
        temp=temp->right;

    if(temp->left==NULL)
        list=NULL;
    else
    {
        q=temp->left;
        q->right=NULL;
    }

    printf("%d\n", temp->info);
    temp->left=NULL;
    temp->right=NULL;
    free(temp);
}

void delete_node(NODEPTR p)
{
    NODEPTR l, r;
    int x;
    if(p==NULL)
    {
        printf("Invalid deletion\n");
        return;
    }
}

```

```

x = p->info;
l = p->left;
r = p->right;

if(l != NULL)
    l->right = r;
else
    list = r; // deleted head

if(r != NULL)
    r->left = l;

free(p);
printf("The deleted element = %d\n", x);
}

```

```

void display()
{
    temp=list;
    if(temp==NULL)
    {
        printf("No nodes existed\n");
        return;
    }
    while(temp!=NULL)
    {
        printf("%d\t", temp->info);
        temp = temp->right;
    }
    printf("\n");
}

```

3. Write a C program to perform realization of Static Stack.

```

#include <stdio.h>
#define STACKSIZE 4
#define TRUE 1
#define FALSE 0

```

```
struct stack
{
    int top;
    int item[STACKSIZE];
};
```

```
struct stack s;
```

```
void push(int);
void pop();
int empty();
```

```
int main()
{
    char ch;
    int c, x, i;
    s.top = -1;
    do
    {
        printf("MENU\n");
        printf("1. PUSH\n2. POP\n3. Display\n");
        scanf("%d", &c);
        switch(c)
        {
            case 1:
                printf("Enter the number to be inserted: ");
                scanf("%d", &x);
                push(x);
                break;
            case 2:
                pop();
                break;
            case 3:
                printf("Elements are: ");
                for(i = s.top; i >= 0; i--)
                {
```



```

        printf("%d ", s.item[i]);
    }
    printf("\n");
    break;
default:
    printf("Invalid choice\n");
}
printf("Do you wish to continue (y/n)? ");
scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

void push(int x)
{
    if(s.top == STACKSIZE - 1)
    {
        printf("Stack overflow\n");
    }
    else
    {
        s.item[++s.top] = x;
    }
}

void pop()
{
    if(empty())
    {
        printf("Stack underflow\n");
        return;
    }
    printf("Deleted element is %d\n", s.item[s.top--]);
}

int empty()

```

```
{  
    return (s.top == -1) ? TRUE : FALSE;  
}
```

4. Write a C program to perform realization of Static Queue

```
#include <stdio.h>  
#define QSIZE 4  
#define TRUE 1  
#define FALSE 0  
  
struct Queue {  
    int rear, front;  
    int item[QSIZE];  
};  
  
struct Queue q;  
  
void insert(int);  
int delete();  
int empty();  
  
int main() {  
    char ch;  
    int c, x, i;  
    q.rear = -1;  
    q.front = 0;  
  
    do {  
        printf("\nMENU\n");  
        printf("1. INSERT\n2. DELETE\n3. DISPLAY\n");  
        scanf("%d", &c);  
  
        switch(c) {  
            case 1:  
                printf("Enter the number to be inserted: ");  
                scanf("%d", &x);
```

```

        insert(x);
        break;
    case 2:
        x = delete();
        if(x != -1)
            printf("Deleted element is %d\n", x);
        break;
    case 3:
        if(empty()) {
            printf("Queue is empty\n");
        } else {
            printf("Elements are: ");
            for(i = q.front; i <= q.rear; i++)
                printf("%d ", q.item[i]);
            printf("\n");
        }
        break;
    default:
        printf("Invalid choice\n");
}
printf("Do you wish to continue (y/n)? ");
scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

void insert(int x) {
    if(q.rear == QSIZE-1) {
        printf("Queue overflow\n");
    } else {
        q.item[++q.rear] = x;
    }
}

int delete() {
    if(empty()) {

```

```

        printf("Queue underflow\n");
        return -1;
    }
    return q.item[q.front++];
}

int empty() {
    return (q.rear < q.front) ? TRUE : FALSE;
}

```

5. Write a C program to implement integer Stack using Circular Singly Linked List [CSLL].

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int info;
    struct node *next;
};

typedef struct node* NODEPTR;
NODEPTR top = NULL; // top of the stack

// Function prototypes
NODEPTR getnode();
void push(int x);
void pop();
int empty();
void display();

int main() {
    char ch;
    int choice, x;

    do {
        printf("\n--- Stack using Circular Singly Linked List ---\n");
    }

```

```

printf("1. PUSH\n2. POP\n3. DISPLAY\n");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Enter element: ");
        scanf("%d", &x);
        push(x);
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    default:
        printf("Invalid choice\n");
}

printf("Do you want to continue (y/n)? ");
scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

// Allocate a new node
NODEPTR getnode() {
    return (NODEPTR)malloc(sizeof(struct node));
}

// Push operation
void push(int x) {
    NODEPTR p = getnode();
    p->info = x;

    if(top == NULL) {

```

```

    p->next = p; // circular link to itself
    top = p;
} else {
    // Insert new node at top and maintain circular link
    NODEPTR last = top;
    while(last->next != top)
        last = last->next;

    p->next = top;
    last->next = p;
    top = p;
}
printf("%d pushed onto stack\n", x);
}

```

// Pop operation

```

void pop() {
    if(empty()) {
        printf("Stack underflow\n");
        return;
    }

```

```

    NODEPTR temp = top;
    int x = temp->info;

```

```

    if(top->next == top) { // only one node
        top = NULL;
    } else {

```

```

        // Find the last node to update circular link
        NODEPTR last = top;
        while(last->next != top)
            last = last->next;
        top = top->next;
        last->next = top;
    }

```

```

    free(temp);

```

```

    printf("Deleted element is %d\n", x);
}

// Check if stack is empty
int empty() {
    return (top == NULL);
}

// Display stack elements
void display() {
    if(empty()) {
        printf("Stack is empty\n");
        return;
    }

    NODEPTR temp = top;
    printf("Stack elements (top to bottom): ");
    do {
        printf("%d ", temp->info);
        temp = temp->next;
    } while(temp != top);
    printf("\n");
}

```

6. Write a C program to implement integer Stack using Circular Doubly Linked List [CDLL] .

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```

struct node {
    int info;

    struct node *left;
    struct node *right;
}

```

```
};
```

```
typedef struct node* NODEPTR;
```

```
NODEPTR top = NULL; // top of the stack
```

```
// Function prototypes
```

```
NODEPTR getnode();
```

```
void push(int x);
```

```
void pop();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    int choice, x;
```

```
    char ch;
```

```
    do {
```

```
        printf("\n--- Stack using Circular Doubly Linked List ---\n");
```

```
        printf("1. PUSH\n2. POP\n3. DISPLAY\n");
```

```
        scanf("%d", &choice);
```

```
        switch(choice) {
```

```
            case 1:
```

```
                printf("Enter element to push: ");
```

```
                scanf("%d", &x);
```



```
        push(x);
        break;
    case 2:
        pop();
        break;
    case 3:
        display();
        break;
    default:
        printf("Invalid choice\n");
}
```

```
    printf("Do you want to continue (y/n)? ");
    scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');
```

```
    return 0;
}
```

// Allocate new node

```
NODEPTR getnode() {
    return (NODEPTR)malloc(sizeof(struct node));
}
```

// Push operation (insert at top)

```
void push(int x) {
```

```

NODEPTR p = getnode();
p->info = x;

if(top == NULL) {
    p->left = p->right = p; // points to itself (circular)
    top = p;
} else {
    NODEPTR last = top->left; // last node in CDLL
    p->right = top;
    p->left = last;
    last->right = p;
    top->left = p;
    top = p; // new top
}
printf("%d pushed onto stack\n", x);
}

```

// Pop operation (delete from top)

```

void pop() {
    if(empty()) {
        printf("Stack underflow\n");
        return;
    }

```

```

NODEPTR temp = top;

```

```

int x = temp->info;

```

```

if(top->right == top) { // only one node
    top = NULL;
} else {
    NODEPTR last = top->left;
    NODEPTR next = top->right;
    last->right = next;
    next->left = last;
    top = next;
}

free(temp);
printf("Deleted element is %d\n", x);
}

// Check if stack is empty
int empty() {
    return (top == NULL);
}

// Display stack elements from top to bottom
void display() {
    if(empty()) {
        printf("Stack is empty\n");
        return;
    }

```

```

NODEPTR temp = top;
printf("Stack elements (top to bottom): ");
do {
    printf("%d ", temp->info);
    temp = temp->right;
} while(temp != top);
printf("\n");
}

```

7. Write a C program to implement integer Queue using CSLL.

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct node {
    int info;
    struct node *next;
};

```

```

typedef struct node* NODEPTR;

```

```

NODEPTR front = NULL; // front of the queue

```

```

NODEPTR rear = NULL; // rear of the queue

```

```

// Function prototypes

```

```

NODEPTR getnode();

```

```

void insert(int x); // enqueue

```

```
void delete();    // dequeue
int empty();
void display();

int main() {
    char ch;
    int choice, x;

    do {
        printf("\n--- Queue using Circular Singly Linked List ---\n");
        printf("1. INSERT (Enqueue)\n2. DELETE (Dequeue)\n3. DISPLAY\n");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter element to insert: ");
                scanf("%d", &x);
                insert(x);
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            default:
```

```

        printf("Invalid choice\n");
    }

    printf("Do you wish to continue (y/n)? ");
    scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

// Allocate a new node
NODEPTR getnode() {
    return (NODEPTR)malloc(sizeof(struct node));
}

// Insert (enqueue) at rear
void insert(int x) {
    NODEPTR p = getnode();
    p->info = x;

    if(empty()) {
        front = rear = p;
        p->next = p; // circular link to itself
    } else {
        p->next = front;
        rear->next = p;
    }
}

```

```

        rear = p;
    }

    printf("%d inserted into queue\n", x);
}

// Delete (dequeue) from front
void delete() {
    if(empty()) {
        printf("Queue underflow\n");
        return;
    }

    NODEPTR temp = front;
    int x = temp->info;

    if(front == rear) { // only one node
        front = rear = NULL;
    } else {
        front = front->next;
        rear->next = front; // maintain circular link
    }

    free(temp);
    printf("Deleted element is %d\n", x);
}

```

```

// Check if queue is empty
int empty() {
    return (front == NULL);
}

// Display queue elements from front to rear
void display() {
    if(empty()) {
        printf("Queue is empty\n");
        return;
    }

    NODEPTR temp = front;
    printf("Queue elements (front to rear): ");
    do {
        printf("%d ", temp->info);
        temp = temp->next;
    } while(temp != front);
    printf("\n");
}

```

8. Write a C program to implement integer Queue using CDLL

```

#include <stdio.h>
#include <stdlib.h>

// Node structure

```



```
struct node {  
    int info;  
    struct node *left;  
    struct node *right;  
};
```

```
typedef struct node* NODEPTR;
```

```
NODEPTR front = NULL; // front of the queue
```

```
NODEPTR rear = NULL; // rear of the queue
```

```
// Function prototypes
```

```
NODEPTR getnode();
```

```
void enqueue(int x);
```

```
void dequeue();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    int choice, x;
```

```
    char ch;
```

```
    do {
```

```
        printf("\n--- Queue using Circular Doubly Linked List ---\n");
```

```
        printf("1. INSERT (Enqueue)\n2. DELETE (Dequeue)\n3. DISPLAY\n");
```

```
        scanf("%d", &choice);
```

```

switch(choice) {
    case 1:
        printf("Enter element to insert: ");
        scanf("%d", &x);
        enqueue(x);
        break;
    case 2:
        dequeue();
        break;
    case 3:
        display();
        break;
    default:
        printf("Invalid choice\n");
}

printf("Do you wish to continue (y/n)? ");
scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

// Allocate a new node
NODEPTR getnode() {

```

```
    return (NODEPTR)malloc(sizeof(struct node));  
}
```

// Enqueue at rear

```
void enqueue(int x) {  
    NODEPTR p = getnode();  
    p->info = x;  
  
    if(empty()) {  
        p->left = p->right = p; // circular link  
        front = rear = p;  
    } else {  
        p->right = front;  
        p->left = rear;  
        rear->right = p;  
        front->left = p;  
        rear = p;  
    }  
    printf("%d inserted into queue\n", x);  
}
```

// Dequeue from front

```
void dequeue() {  
    if(empty()) {  
        printf("Queue underflow\n");  
        return;  
    }  
}
```

```
}
```

```
NODEPTR temp = front;
```

```
int x = temp->info;
```

```
if(front == rear) { // only one node
```

```
    front = rear = NULL;
```

```
} else {
```

```
    front = front->right;
```

```
    front->left = rear;
```

```
    rear->right = front;
```

```
}
```

```
free(temp);
```

```
printf("Deleted element is %d\n", x);
```

```
}
```

```
// Check if queue is empty
```

```
int empty() {
```

```
    return (front == NULL);
```

```
}
```

```
// Display queue from front to rear
```

```
void display() {
```

```
    if(empty()) {
```

```
        printf("Queue is empty\n");
```

```

        return;
    }

    NODEPTR temp = front;
    printf("Queue elements (front to rear): ");
    do {
        printf("%d ", temp->info);
        temp = temp->right;
    } while(temp != front);
    printf("\n");
}

```

9. Write a C program to implement Employee Stack using SLL. Employee Details - Employee No, Employee name, Basic, Designation, Date of Joining.

```

#include <stdio.h>
#include <stdlib.h>
#define TRUE 1
#define FALSE 0

struct employee {
    int empNo;
    char name[50];
    float basic;
    char designation[30];
    int doj_day;
    int doj_month;
    int doj_year;
}

```

```
    struct employee *next;  
};
```

```
struct employee *top = NULL;
```

```
// Function prototypes
```

```
void push();
```

```
void pop();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    char ch;
```

```
    int choice;
```

```
    do {
```

```
        printf("\n--- Employee Stack Menu ---\n");
```

```
        printf("1. PUSH (Add Employee)\n2. POP (Remove Employee)\n3.  
Display Stack\n");
```

```
        scanf("%d", &choice);
```

```
        switch(choice) {
```

```
            case 1:
```

```
                push();
```

```
                break;
```

```
            case 2:
```

```

        pop();
        break;
    case 3:
        display();
        break;
    default:
        printf("Invalid choice\n");
    }

    printf("Do you wish to continue (y/n)? ");
    scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

// Push operation
void push() {
    struct employee *p = (struct employee*)malloc(sizeof(struct employee));

    printf("Enter Employee No: ");
    scanf("%d", &p->empNo);
    printf("Enter Name: ");
    scanf(" %[^\\n]", p->name);
    printf("Enter Basic Salary: ");
    scanf("%f", &p->basic);

```

```

printf("Enter Designation: ");
scanf(" %[^\\n]", p->designation);
printf("Enter Date of Joining (dd mm yyyy): ");
scanf("%d %d %d", &p->doj_day, &p->doj_month, &p->doj_year);

p->next = top;
top = p;

printf("Employee pushed onto stack successfully.\\n");
}

// Pop operation
void pop() {
    if(empty()) {
        printf("Stack underflow. No employee to pop.\\n");
        return;
    }

    struct employee *temp = top;
    printf("Deleted Employee Details:\\n");
    printf("Employee No: %d\\n", temp->empNo);
    printf("Name: %s\\n", temp->name);
    printf("Basic: %.2f\\n", temp->basic);
    printf("Designation: %s\\n", temp->designation);
    printf("Date of Joining: %02d-%02d-%04d\\n", temp->doj_day,
temp->doj_month, temp->doj_year);

```



```

    top = top->next;
    free(temp);
}

// Check if stack is empty
int empty() {
    return (top == NULL) ? TRUE : FALSE;
}

// Display all employees
void display() {
    if(empty()) {
        printf("Stack is empty.\n");
        return;
    }

    struct employee *p = top;
    printf("\n--- Employee Stack (Top to Bottom) ---\n");
    while(p != NULL) {
        printf("Employee No: %d\n", p->empNo);
        printf("Name: %s\n", p->name);
        printf("Basic: %.2f\n", p->basic);
        printf("Designation: %s\n", p->designation);
        printf("Date of Joining: %02d-%02d-%04d\n\n", p->doj_day,
p->doj_month, p->doj_year);
    }
}

```

```
    p = p->next;
}
}
```

**10. Write a C program to implement Employee Stack using DLL.
Employee Details - Employee No, Employee name, Basic, Designation, Date of Joining.**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct employee {
    int empNo;
    char name[50];
    float basic;
    char designation[30];
    char doj[12]; // Date of Joining as a single string
    struct employee *left;
    struct employee *right;
};
```

```
struct employee *top = NULL; // top of stack
```

```
// Function prototypes
```

```
void push();
void pop();
int empty();
void display();
```

```
int main() {  
    char ch;  
    int choice;  
  
    do {  
        printf("\n--- Employee Stack Menu ---\n");  
        printf("1. PUSH\n2. POP\n3. Display\n");  
        scanf("%d", &choice);  
  
        switch(choice) {  
            case 1:  
                push();  
                break;  
            case 2:  
                pop();  
                break;  
            case 3:  
                display();  
                break;  
            default:  
                printf("Invalid choice\n");  
        }  
  
        printf("Do you wish to continue (y/n)? ");  
        scanf(" %c", &ch);  
    }  
}
```

```

    } while(ch != 'n' && ch != 'N');

    return 0;
}

// Push operation (insert at top)
void push() {
    struct employee *p = (struct employee*)malloc(sizeof(struct employee));

    printf("Enter Employee No: ");
    scanf("%d", &p->empNo);
    printf("Enter Name: ");
    scanf(" %[^\\n]", p->name);
    printf("Enter Basic Salary: ");
    scanf("%f", &p->basic);
    printf("Enter Designation: ");
    scanf(" %[^\\n]", p->designation);
    printf("Enter Date of Joining (dd-mm-yyyy): ");
    scanf(" %[^\\n]", p->doj);

    p->left = NULL;
    p->right = top;
    if(top != NULL)
        top->left = p;
    top = p;
}

```

```
    printf("Employee pushed onto stack successfully.\n");  
}
```

// Pop operation (delete from top)

```
void pop() {  
    if(empty()) {  
        printf("Stack underflow.\n");  
        return;  
    }  
}
```

```
struct employee *temp = top;
```

```
printf("Deleted Employee Details:\n");  
printf("Employee No: %d\n", temp->empNo);  
printf("Name: %s\n", temp->name);  
printf("Basic: %.2f\n", temp->basic);  
printf("Designation: %s\n", temp->designation);  
printf("Date of Joining: %s\n", temp->doj);
```

```
top = top->right;  
if(top != NULL)  
    top->left = NULL;
```

```
free(temp);  
}
```

```

// Check if stack is empty
int empty() {
    return (top == NULL);
}

// Display all employees from top to bottom
void display() {
    if(empty()) {
        printf("Stack is empty.\n");
        return;
    }

    struct employee *p = top;
    printf("\n--- Employee Stack (Top to Bottom) ---\n");
    while(p != NULL) {
        printf("Employee No: %d\n", p->empNo);
        printf("Name: %s\n", p->name);
        printf("Basic: %.2f\n", p->basic);
        printf("Designation: %s\n", p->designation);
        printf("Date of Joining: %s\n\n", p->doj);
        p = p->right;
    }
}

```

11. Write a C program to implement Employee Queue using SLL. Employee Details - Employee No, Employee name, Basic, Designation, Date of Joining.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct employee {
    int empNo;
    char name[50];
    float basic;
    char designation[30];
    char doj[12]; // Date of Joining as a single string "dd-mm-yyyy"
    struct employee *next;
};
```

```
struct employee *front = NULL; // front of queue
struct employee *rear = NULL; // rear of queue
```

```
// Function prototypes
```

```
void enqueue();
void dequeue();
int empty();
void display();
```

```
int main() {
    char ch;
    int c;
```

```
do {  
    printf("\n--- Employee Queue Menu ---\n");  
    printf("1. ENQUEUE (Insert Employee)\n");  
    printf("2. DEQUEUE (Remove Employee)\n");  
    printf("3. Display Queue\n");  
    scanf("%d", &c);  
  
    switch(c) {  
        case 1:  
            enqueue();  
            break;  
        case 2:  
            dequeue();  
            break;  
        case 3:  
            display();  
            break;  
        default:  
            printf("Invalid choice\n");  
    }  
  
    printf("Do you wish to continue (y/n)? ");  
    scanf(" %c", &ch);  
} while(ch != 'n' && ch != 'N');
```



```

    return 0;
}

// Check if queue is empty
int empty() {
    return (front == NULL);
}

// Insert employee at rear
void enqueue() {
    struct employee *p = (struct employee*)malloc(sizeof(struct employee));

    printf("Enter Employee No: ");
    scanf("%d", &p->empNo);
    printf("Enter Name: ");
    scanf(" %s", p->name);
    printf("Enter Basic Salary: ");
    scanf("%f", &p->basic);
    printf("Enter Designation: ");
    scanf(" %s", p->designation);
    printf("Enter Date of Joining (dd-mm-yyyy): ");
    scanf(" %s", p->doj);

    p->next = NULL;

    if(empty()) {

```

```

        front = rear = p;
    } else {
        rear->next = p;
        rear = p;
    }

    printf("Employee enqueued successfully.\n");
}

// Remove employee from front
void dequeue() {
    if(empty()) {
        printf("Queue underflow. No employee to dequeue.\n");
        return;
    }

    struct employee *temp = front;

    printf("Deleted Employee Details:\n");
    printf("Employee No: %d\n", temp->empNo);
    printf("Name: %s\n", temp->name);
    printf("Basic: %.2f\n", temp->basic);
    printf("Designation: %s\n", temp->designation);
    printf("Date of Joining: %s\n", temp->doj);

    front = front->next;

```

```

    if(front == NULL) // queue became empty
        rear = NULL;

    free(temp);
}

// Display all employees in queue
void display() {
    if(empty()) {
        printf("Queue is empty.\n");
        return;
    }

    struct employee *p = front;
    printf("\n--- Employee Queue (Front to Rear) ---\n");
    while(p != NULL) {
        printf("Employee No: %d\n", p->empNo);
        printf("Name: %s\n", p->name);
        printf("Basic: %.2f\n", p->basic);
        printf("Designation: %s\n", p->designation);
        printf("Date of Joining: %s\n\n", p->doj);
        p = p->next;
    }
}

```

12. Write a C program to implement Employee Queue using DLL. Employee Details - Employee No, Employee name, Basic, Designation, Date of Joining.

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct employee {
    int empNo;
    char name[50];
    float basic;
    char designation[30];
    char doj[12]; // Date of Joining as a single string
    struct employee *left;
    struct employee *right;
};

struct employee *front = NULL; // front of queue
struct employee *rear = NULL; // rear of queue

// Function prototypes
void enqueue();
void dequeue();
int empty();
void display();
```

```
int main() {  
    char ch;  
    int c;  
  
    do {  
        printf("\n--- Employee Queue using DLL ---\n");  
        printf("1. ENQUEUE (Insert Employee)\n");  
        printf("2. DEQUEUE (Remove Employee)\n");  
        printf("3. Display Queue\n");  
        scanf("%d", &c);  
  
        switch(c) {  
            case 1:  
                enqueue();  
                break;  
            case 2:  
                dequeue();  
                break;  
            case 3:  
                display();  
                break;  
            default:  
                printf("Invalid choice\n");  
        }  
  
        printf("Do you wish to continue (y/n)? ");
```

```

        scanf(" %c", &ch);
    } while(ch != 'n' && ch != 'N');

    return 0;
}

// Check if queue is empty
int empty() {
    return (front == NULL);
}

// Insert employee at rear
void enqueue() {
    struct employee *p = (struct employee*)malloc(sizeof(struct employee));

    printf("Enter Employee No: ");
    scanf("%d", &p->empNo);
    printf("Enter Name: ");
    scanf(" %[^\n]", p->name);
    printf("Enter Basic Salary: ");
    scanf("%f", &p->basic);
    printf("Enter Designation: ");
    scanf(" %[^\n]", p->designation);
    printf("Enter Date of Joining (dd-mm-yyyy): ");
    scanf(" %[^\n]", p->doj);

```

```
p->right = NULL;
```

```
p->left = rear;
```

```
if(empty()) {
```

```
    front = rear = p;
```

```
} else {
```

```
    rear->right = p;
```

```
    rear = p;
```

```
}
```

```
printf("Employee enqueued successfully.\n");
```

```
}
```

```
// Remove employee from front
```

```
void dequeue() {
```

```
    if(empty()) {
```

```
        printf("Queue underflow. No employee to dequeue.\n");
```

```
        return;
```

```
}
```

```
struct employee *temp = front;
```

```
printf("Deleted Employee Details:\n");
```

```
printf("Employee No: %d\n", temp->empNo);
```

```
printf("Name: %s\n", temp->name);
```

```
printf("Basic: %.2f\n", temp->basic);
```

```

printf("Designation: %s\n", temp->designation);
printf("Date of Joining: %s\n", temp->doj);

front = front->right;
if(front != NULL)
    front->left = NULL;
else
    rear = NULL; // queue became empty

free(temp);
}

// Display all employees from front to rear
void display() {
    if(empty()) {
        printf("Queue is empty.\n");
        return;
    }

    struct employee *p = front;
    printf("\n--- Employee Queue (Front to Rear) ---\n");
    while(p != NULL) {
        printf("Employee No: %d\n", p->empNo);
        printf("Name: %s\n", p->name);
        printf("Basic: %.2f\n", p->basic);
        printf("Designation: %s\n", p->designation);
    }
}

```



```

        printf("Date of Joining: %s\n\n", p->doj);
        p = p->right;
    }
}

```

13. Write a C program to implement Student Stack using SLL. Student details – Reg. No, Student Name, Semester, Marks.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

struct student {
    int regNo;
    char name[50];
    int semester;
    int marks;
    struct student *next;
};

```

```

struct student *top = NULL;

```

```

void push();
void pop();
int empty();
void display();

```

```

int main() {
    char ch;
    int c;

    do {
        printf("\n--- Student Stack using SLL ---\n");
        printf("1. PUSH\n2. POP\n3. Display\n");
        scanf("%d", &c);

        switch(c) {
            case 1: push(); break;

```

```

        case 2: pop(); break;
        case 3: display(); break;
        default: printf("Invalid choice\n");
    }

    printf("Do you wish to continue (y/n)? ");
    scanf(" %c", &ch);
} while(ch != 'n' && ch != 'N');

return 0;
}

void push() {
    struct student *p = (struct student*)malloc(sizeof(struct student));

    printf("Enter Reg. No: ");
    scanf("%d", &p->regNo);
    printf("Enter Name: ");
    scanf(" %[^\\n]", p->name);
    printf("Enter Semester: ");
    scanf("%d", &p->semester);
    printf("Enter Marks: ");
    scanf("%d", &p->marks);

    p->next = top;
    top = p;
}

void pop() {
    if(empty()) {
        printf("Stack underflow.\\n");
        return;
    }

    struct student *temp = top;
    printf("Deleted Student:\\nReg. No: %d\\nName: %s\\nSemester: %d\\nMarks: %d\\n",
        temp->regNo, temp->name, temp->semester, temp->marks);

    top = top->next;

```

```

    free(temp);
}

int empty() {
    return (top == NULL);
}

void display() {
    if(empty()) {
        printf("Stack is empty.\n");
        return;
    }

    struct student *p = top;
    printf("\n--- Student Stack (Top to Bottom) ---\n");
    while(p != NULL) {
        printf("Reg. No: %d, Name: %s, Semester: %d, Marks: %d\n",
            p->regNo, p->name, p->semester, p->marks);
        p = p->next;
    }
}

```

14. Write a C program to implement Student Stack using **DLL. Student details – Reg. No, Student Name, Semester, Marks.**

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct student {
    int regNo;
    char name[50];
    int semester;
    int marks;
    struct student *left;

```

```
    struct student *right;  
};
```

```
struct student *top = NULL;
```

```
void push();
```

```
void pop();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    char ch;
```

```
    int c;
```

```
    do {
```

```
        printf("\n--- Student Stack using DLL ---\n");
```

```
        printf("1. PUSH\n2. POP\n3. Display\n");
```

```
        scanf("%d", &c);
```

```
        switch(c) {
```

```
            case 1: push(); break;
```

```
            case 2: pop(); break;
```

```
            case 3: display(); break;
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```

        printf("Do you wish to continue (y/n)? ");
        scanf(" %c", &ch);
    } while(ch != 'n' && ch != 'N');

    return 0;
}

void push() {
    struct student *p = (struct student*)malloc(sizeof(struct student));

    printf("Enter Reg. No: ");
    scanf("%d", &p->regNo);
    printf("Enter Name: ");
    scanf(" %[^\\n]", p->name);
    printf("Enter Semester: ");
    scanf("%d", &p->semester);
    printf("Enter Marks: ");
    scanf("%d", &p->marks);

    p->left = NULL;
    p->right = top;
    if(top != NULL)
        top->left = p;
    top = p;
}

```

```
void pop() {  
    if(empty()) {  
        printf("Stack underflow.\n");  
        return;  
    }  
}
```

```
    struct student *temp = top;  
    printf("Deleted Student:\nReg. No: %d\nName: %s\nSemester: %d\nMarks:  
%d\n",  
        temp->regNo, temp->name, temp->semester, temp->marks);
```

```
    top = top->right;  
    if(top != NULL)  
        top->left = NULL;
```

```
    free(temp);  
}
```

```
int empty() {  
    return (top == NULL);  
}
```

```
void display() {  
    if(empty()) {  
        printf("Stack is empty.\n");  
        return;  
    }  
}
```

```

    }

    struct student *p = top;
    printf("\n--- Student Stack (Top to Bottom) ---\n");
    while(p != NULL) {
        printf("Reg. No: %d, Name: %s, Semester: %d, Marks: %d\n",
            p->regNo, p->name, p->semester, p->marks);
        p = p->right;
    }
}

```

15. Write a C program to implement Student Queue using SLL. Student details – Reg. No, Student Name, Semester, Marks.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct student {
    int regNo;
    char name[50];
    int semester;
    int marks;
    struct student *next;
};

struct student *front = NULL;
struct student *rear = NULL;

```

```
void enqueue();
```

```
void dequeue();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    char ch;
```

```
    int c;
```

```
    do {
```

```
        printf("\n--- Student Queue using SLL ---\n");
```

```
        printf("1. ENQUEUE\n2. DEQUEUE\n3. Display\n");
```

```
        scanf("%d", &c);
```

```
        switch(c) {
```

```
            case 1: enqueue(); break;
```

```
            case 2: dequeue(); break;
```

```
            case 3: display(); break;
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```
        printf("Do you wish to continue (y/n)? ");
```

```
        scanf(" %c", &ch);
```

```
    } while(ch != 'n' && ch != 'N');
```



```
    return 0;
}
```

```
int empty() {
    return (front == NULL);
}
```

```
void enqueue() {
    struct student *p = (struct student*)malloc(sizeof(struct student));
```

```
    printf("Enter Reg. No: ");
    scanf("%d", &p->regNo);
    printf("Enter Name: ");
    scanf(" %[^\n]", p->name);
    printf("Enter Semester: ");
    scanf("%d", &p->semester);
    printf("Enter Marks: ");
    scanf("%d", &p->marks);
```

```
    p->next = NULL;
```

```
    if(empty())
        front = rear = p;
    else {
        rear->next = p;
        rear = p;
```

```
}  
}
```

```
void dequeue() {  
    if(empty()) {  
        printf("Queue underflow.\n");  
        return;  
    }  
}
```

```
    struct student *temp = front;  
    printf("Deleted Student:\nReg. No: %d\nName: %s\nSemester: %d\nMarks:  
%d\n",  
        temp->regNo, temp->name, temp->semester, temp->marks);
```

```
    front = front->next;  
    if(front == NULL)  
        rear = NULL;  
  
    free(temp);  
}
```

```
void display() {  
    if(empty()) {  
        printf("Queue is empty.\n");  
        return;  
    }  
}
```

```

struct student *p = front;
printf("\n--- Student Queue (Front to Rear) ---\n");
while(p != NULL) {
    printf("Reg. No: %d, Name: %s, Semester: %d, Marks: %d\n",
        p->regNo, p->name, p->semester, p->marks);
    p = p->next;
}
}

```

16. Write a C program to implement Student Queue using DLL. Student details – Reg. No, Student Name, Semester, Marks.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

struct student {
    int regNo;
    char name[50];
    int semester;
    int marks;
    struct student *left;
    struct student *right;
};

```

```

struct student *front = NULL;
struct student *rear = NULL;

```

```
void enqueue();
```

```
void dequeue();
```

```
int empty();
```

```
void display();
```

```
int main() {
```

```
    char ch;
```

```
    int c;
```

```
    do {
```

```
        printf("\n--- Student Queue using DLL ---\n");
```

```
        printf("1. ENQUEUE\n2. DEQUEUE\n3. Display\n");
```

```
        scanf("%d", &c);
```

```
        switch(c) {
```

```
            case 1: enqueue(); break;
```

```
            case 2: dequeue(); break;
```

```
            case 3: display(); break;
```

```
            default: printf("Invalid choice\n");
```

```
        }
```

```
        printf("Do you wish to continue (y/n)? ");
```

```
        scanf(" %c", &ch);
```

```
    } while(ch != 'n' && ch != 'N');
```

```
    return 0;
}
```

```
int empty() {
    return (front == NULL);
}
```

```
void enqueue() {
    struct student *p = (struct student*)malloc(sizeof(struct student));

    printf("Enter Reg. No: ");
    scanf("%d", &p->regNo);
    printf("Enter Name: ");
    scanf(" %[^\\n]", p->name);
    printf("Enter Semester: ");
    scanf("%d", &p->semester);
    printf("Enter Marks: ");
    scanf("%d", &p->marks);

    p->right = NULL;
    p->left = rear;

    if(empty())
        front = rear = p;
    else {
        rear->right = p;
```

```
        rear = p;
    }
}
```

```
void dequeue() {
    if(empty()) {
        printf("Queue underflow.\n");
        return;
    }
```

```
    struct student *temp = front;

    printf("Deleted Student:\nReg. No: %d\nName: %s\nSemester: %d\nMarks:
%d\n",
        temp->regNo, temp->name, temp->semester, temp->marks);
```

```
    front = front->right;
    if(front != NULL)
        front->left = NULL;
    else
        rear = NULL;
```

```
    free(temp);
}
```

```
void display() {
    if(empty()) {
```

```
printf("Queue is empty.\n");  
return;  
}
```

```
struct student *p = front;  
printf("\n--- Student Queue (Front to Rear) ---\n");  
while(p != NULL) {  
    printf("Reg. No: %d, Name: %s, Semester: %d, Marks: %d\n",  
        p->regNo, p->name, p->semester, p->marks);  
    p = p->right;  
}  
}
```

17. Write a C program to create a Binary search tree and display the node content using any one traversal technique.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {  
    int info;  
    struct node *left;  
    struct node *right;  
};
```

```
typedef struct node* nodeptr;
```

```
// Function prototypes
```

```
nodeptr getnode();
```

```
nodeptr insert(nodeptr root, int x);
```

```
void inorder(nodeptr root);
```

```
int main() {
```

```
    nodeptr root = NULL;
```

```
    int number;
```

```
    char ch;
```

```
    do {
```

```
        printf("Enter a number to insert into BST: ");
```

```
        scanf("%d", &number);
```

```
        root = insert(root, number);
```

```
        printf("Do you want to continue inserting (y/n)? ");
```

```
        scanf(" %c", &ch);
```

```
    } while(ch == 'y' || ch == 'Y');
```

```
    printf("\nBST nodes in Inorder traversal:\n");
```

```
    inorder(root);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

```
// Create a new node
```



```
nodeptr getnode() {  
    nodeptr p = (nodeptr)malloc(sizeof(struct node));  
    p->left = NULL;  
    p->right = NULL;  
    return p;  
}
```

// Insert a number into BST

```
nodeptr insert(nodeptr root, int x) {  
    if(root == NULL) {  
        nodeptr p = getnode();  
        p->info = x;  
        return p;  
    }  
  
    if(x < root->info)  
        root->left = insert(root->left, x);  
    else if(x > root->info)  
        root->right = insert(root->right, x);  
    else  
        printf("%d is a duplicate value. Ignored.\n", x);  
  
    return root;  
}
```

// Inorder traversal

```

void inorder(nodeptr root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%d\t", root->info);
        inorder(root->right);
    }
}

```

18. Write a C program to implement three tree traversal techniques

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct node {
    int info;
    struct node *left;
    struct node *right;
};

```

```

typedef struct node* nodeptr;

```

```

// Function prototypes

```

```

nodeptr getnode();
nodeptr insert(nodeptr root, int x);
void inorder(nodeptr root);
void preorder(nodeptr root);
void postorder(nodeptr root);

```

```

int main() {
    nodeptr root = NULL;
    int number, choice;
    char ch;

    do {
        printf("Enter a number to insert into BST: ");
        scanf("%d", &number);
        root = insert(root, number);

        printf("Do you want to continue inserting (y/n)? ");
        scanf(" %c", &ch);
    } while(ch == 'y' || ch == 'Y');

    do {
        printf("\n--- Tree Traversal Menu ---\n");
        printf("1. Inorder Traversal\n");
        printf("2. Preorder Traversal\n");
        printf("3. Postorder Traversal\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Inorder: ");

```

```

        inorder(root);
        printf("\n");
        break;
case 2:
    printf("Preorder: ");
    preorder(root);
    printf("\n");
    break;
case 3:
    printf("Postorder: ");
    postorder(root);
    printf("\n");
    break;
case 4:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice!\n");
}
} while(choice != 4);

return 0;
}

// Create a new node
nodeptr getnode() {

```

```
nodeptr p = (nodeptr)malloc(sizeof(struct node));  
p->left = NULL;  
p->right = NULL;  
return p;  
}
```

// Insert into BST

```
nodeptr insert(nodeptr root, int x) {  
    if(root == NULL) {  
        nodeptr p = getnode();  
        p->info = x;  
        return p;  
    }  
  
    if(x < root->info)  
        root->left = insert(root->left, x);  
    else if(x > root->info)  
        root->right = insert(root->right, x);  
    else  
        printf("%d is a duplicate value. Ignored.\n", x);  
  
    return root;  
}
```

// Inorder traversal

```
void inorder(nodeptr root) {
```

```

    if(root != NULL) {
        inorder(root->left);
        printf("%d\t", root->info);
        inorder(root->right);
    }
}

```

// Preorder traversal

```

void preorder(nodeptr root) {
    if(root != NULL) {
        printf("%d\t", root->info);
        preorder(root->left);
        preorder(root->right);
    }
}

```

// Postorder traversal

```

void postorder(nodeptr root) {
    if(root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d\t", root->info);
    }
}

```

19. Write a C program to sort n elements using Binary Tree Sort technique.

```

#include <stdio.h>

```

```
#include <stdlib.h>
```

```
struct node {  
    int info;  
    struct node *left;  
    struct node *right;  
};
```

```
typedef struct node* nodeptr;
```

```
// Function prototypes
```

```
nodeptr getnode();
```

```
nodeptr insert(nodeptr root, int x);
```

```
void inorder(nodeptr root, int arr[], int *index);
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter %d elements:\n", n);
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &arr[i]);
```

```
    nodeptr root = NULL;
```

```

for(i = 0; i < n; i++)
    root = insert(root, arr[i]);

// Retrieve sorted elements using inorder traversal
int index = 0;
inorder(root, arr, &index);

printf("Sorted elements:\n");
for(i = 0; i < n; i++)
    printf("%d\t", arr[i]);
printf("\n");

return 0;
}

// Create a new node
nodeptr getnode() {
    nodeptr p = (nodeptr)malloc(sizeof(struct node));
    p->left = NULL;
    p->right = NULL;
    return p;
}

// Insert into BST
nodeptr insert(nodeptr root, int x) {
    if(root == NULL) {

```



```

    nodeptr p = getnode();
    p->info = x;
    return p;
}

if(x < root->info)
    root->left = insert(root->left, x);
else
    root->right = insert(root->right, x); // duplicates go to right

return root;
}

// Inorder traversal: store elements in array
void inorder(nodeptr root, int arr[], int *index) {
    if(root != NULL) {
        inorder(root->left, arr, index);
        arr[(*index)++] = root->info;
        inorder(root->right, arr, index);
    }
}

```

20. Write a C program to sort n elements using Heap Sort technique

```
#include <stdio.h>
```

```

void heapify(int arr[], int n, int i);
void heapSort(int arr[], int n);

```

```
void display(int arr[], int n);
```

```
int main() {
```

```
    int n, i;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter %d elements:\n", n);
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &arr[i]);
```

```
    printf("Original Array:\n");
```

```
    display(arr, n);
```

```
    heapSort(arr, n);
```

```
    printf("Sorted Array (Ascending using Heap Sort):\n");
```

```
    display(arr, n);
```

```
    return 0;
```

```
}
```

```
// Function to heapify a subtree rooted at index i
```

```
void heapify(int arr[], int n, int i) {
```

```
    int largest = i;    // Initialize largest as root
```

```

int left = 2*i + 1; // left child
int right = 2*i + 2; // right child

// If left child is larger
if(left < n && arr[left] > arr[largest])
    largest = left;

// If right child is larger
if(right < n && arr[right] > arr[largest])
    largest = right;

// If largest is not root
if(largest != i) {
    int temp = arr[i];
    arr[i] = arr[largest];
    arr[largest] = temp;

    // Recursively heapify the affected subtree
    heapify(arr, n, largest);
}
}

// Heap Sort function
void heapSort(int arr[], int n) {
    int i;

    // Build max heap

```

```

for(i = n/2 - 1; i >= 0; i--)
    heapify(arr, n, i);

// Extract elements from heap one by one
for(i = n-1; i > 0; i--) {
    // Move current root to end
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;

    // Call max heapify on the reduced heap
    heapify(arr, i, 0);
}

// Function to display the array
void display(int arr[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d\t", arr[i]);
    printf("\n");
}

```